

Correction — TD/TP de synthèse

Types, classes, RAII et règle de zéro avec Gmsh

MAM4 — Programmation C++

La version complète se trouve dans `code/reference/synthese_gmsh`. Cette correction présente les choix C++ importants ; les appels techniques à Gmsh restent dans l'adaptateur fourni.

Exercice 0 — Premier maillage

`main.cpp` contient le point d'entrée. La fonction `generate_rectangle` reçoit la configuration et génère le fichier. Les appels propres à Gmsh sont regroupés dans `gmsh_adapter.cpp` et, pour l'initialisation de la bibliothèque, dans `gmsh_session.cpp`.

Exercice 1 — Signatures

```
void generate_rectangle(const MeshConfig& config);
void refine(MeshConfig& config, double mesh_size);
void print_config(const MeshConfig& config);
```

Réponse. `generate_rectangle` et `print_config` observent la configuration : elles utilisent `const MeshConfig&`. `refine` doit modifier l'objet de l'appelant : elle utilise `MeshConfig&`.

Avec `const MeshConfig&`, l'affectation à un membre est refusée : la fonction a promis de ne pas modifier l'objet par cette référence.

Exercice 2 — Une configuration correcte

L'invariant retenu est :

`width_ > 0, height_ > 0, mesh_size_ > 0, output_ non vide.`

Le fichier `mesh_config.hpp` contient :

```
#pragma once
#include <string>
class MeshConfig {
public:
    MeshConfig(double width, double height,
               double mesh_size, std::string output);
    double width() const noexcept;
    double height() const noexcept;
    double mesh_size() const noexcept;
    const std::string& output() const noexcept;
    void set_mesh_size(double mesh_size);
    void set_output(std::string output);
private:
    double width_;
    double height_;
    double mesh_size_;
    std::string output_;
};

void refine(MeshConfig& config, double mesh_size);
void print_config(const MeshConfig& config);
```

Une fonction locale évite de répéter les contrôles :

L'implémentation inclut notamment `<stdexcept>` et `<utility>`.

```
namespace {
double positive(double value, const char* name) {
    if (value > 0.0) return value;
    throw std::invalid_argument{std::string{name}
                                + " must be positive"};
}
std::string non_empty(std::string value) {
    if (!value.empty()) return value;
    throw std::invalid_argument{"empty output name"};
}
}
```

Le constructeur et les setters utilisent ces vérifications :

```
MeshConfig::MeshConfig(double width, double height,
                      double mesh_size, std::string output)
: width_{positive(width, "width")},
  height_{positive(height, "height")},
  mesh_size_{positive(mesh_size, "mesh_size")},
  output_{non_empty(std::move(output))}
{}

void MeshConfig::set_mesh_size(double mesh_size)
{
    mesh_size_ = positive(mesh_size, "mesh_size");
}

void MeshConfig::set_output(std::string output)
{
    output_ = non_empty(std::move(output));
}
```

Les accesseurs renvoient simplement les membres correspondants. Les deux fonctions libres utilisent maintenant l'interface publique :

```
void refine(MeshConfig& config, double mesh_size)
{
    config.set_mesh_size(mesh_size);
}

void print_config(const MeshConfig& config)
{
    std::cout << config.output() << ": "
               << config.width() << " x " << config.height()
               << ", h = " << config.mesh_size() << '\n';
}
```

L'adaptateur remplace de même `config.width` par `config.width()`, et ainsi de suite.

Réponse. `private` empêche les modifications directes depuis l'extérieur. Il ne vérifie pas les valeurs à notre place : le développeur doit maintenir l'invariant dans le constructeur et les setters.

Exercice 3 — Premier rectangle

```
MeshConfig config{2.0, 1.0, 0.25, "coarse.msh"};
print_config(config);
generate_rectangle(config);
```

Réponse. `generate_rectangle` reçoit `const MeshConfig&` car elle observe une configuration existante. La référence évite une copie et `const` interdit sa modification par la fonction.

Exercice 4 — Session Gmsh et RAII

```
class GmshSession {
public:
    GmshSession();
    ~GmshSession();

    GmshSession(const GmshSession&) = delete;
    GmshSession& operator=(const GmshSession&) = delete;
};
```

```
#include "gmsh_session.hpp"
```

```
#include <gmsh.h>
```

```
GmshSession::GmshSession() { gmsh::initialize(); }
GmshSession::~GmshSession() { gmsh::finalize(); }
```

Réponse. Le destructeur est appelé automatiquement à la sortie de la portée. Il garantit l'appel à `gmsh::finalize()`, y compris lorsqu'une fonction quitte sa portée plus tôt. C'est le RAII.

Réponse. `MeshConfig` représente des données que l'on souhaite dupliquer. `GmshSession` contrôle une session globale initialisée puis finalisée : en créer une copie n'a pas de sens, donc la copie est interdite.

Exercice 5 — Deux configurations indépendantes

Le programme final est :

```
#include "gmsh_adapter.hpp"
#include "gmsh_session.hpp"
#include "mesh_config.hpp"

#include <cassert>

int main()
{
    GmshSession session;

    MeshConfig coarse{2.0, 1.0, 0.25, "coarse.msh"};
    MeshConfig fine{coarse};

    refine(fine, 0.08);
    fine.set_output("fine.msh");

    assert(coarse.mesh_size() == 0.25);
    assert(coarse.output() == "coarse.msh");
    assert(fine.mesh_size() == 0.08);
    assert(fine.output() == "fine.msh");

    print_config(coarse);
    print_config(fine);
    generate_rectangle(coarse);
    generate_rectangle(fine);
}
```

Les deux fichiers décrivent un rectangle de dimensions 2×1 . Le second contient un maillage plus fin.

Réponse. Aucun constructeur de copie personnalisé n'est nécessaire. `MeshConfig` contient trois `double` et une `std::string`, qui savent déjà se copier, se déplacer et se détruire. C'est la règle de zéro.

Exercice 6 — Bilan

Notion	Exemple dans le programme
<code>const T&</code>	<code>generate_rectangle</code> , <code>print_config</code>
<code>T&</code>	<code>refine</code>
private et invariant	membres de <code>MeshConfig</code>
constructeur	création d'une configuration valide
destructeur et RAII	<code>GmshSession</code> et <code>finalize</code>
copie indépendante	<code>MeshConfig fine{coarse}</code>
copie interdite	les deux déclarations = <code>delete</code>
règle de zéro	membres valeurs de <code>MeshConfig</code>

À retenir. `MeshConfig` se copie comme une valeur indépendante. `GmshSession` représente au contraire une responsabilité unique. Les types employés expriment directement cette différence.